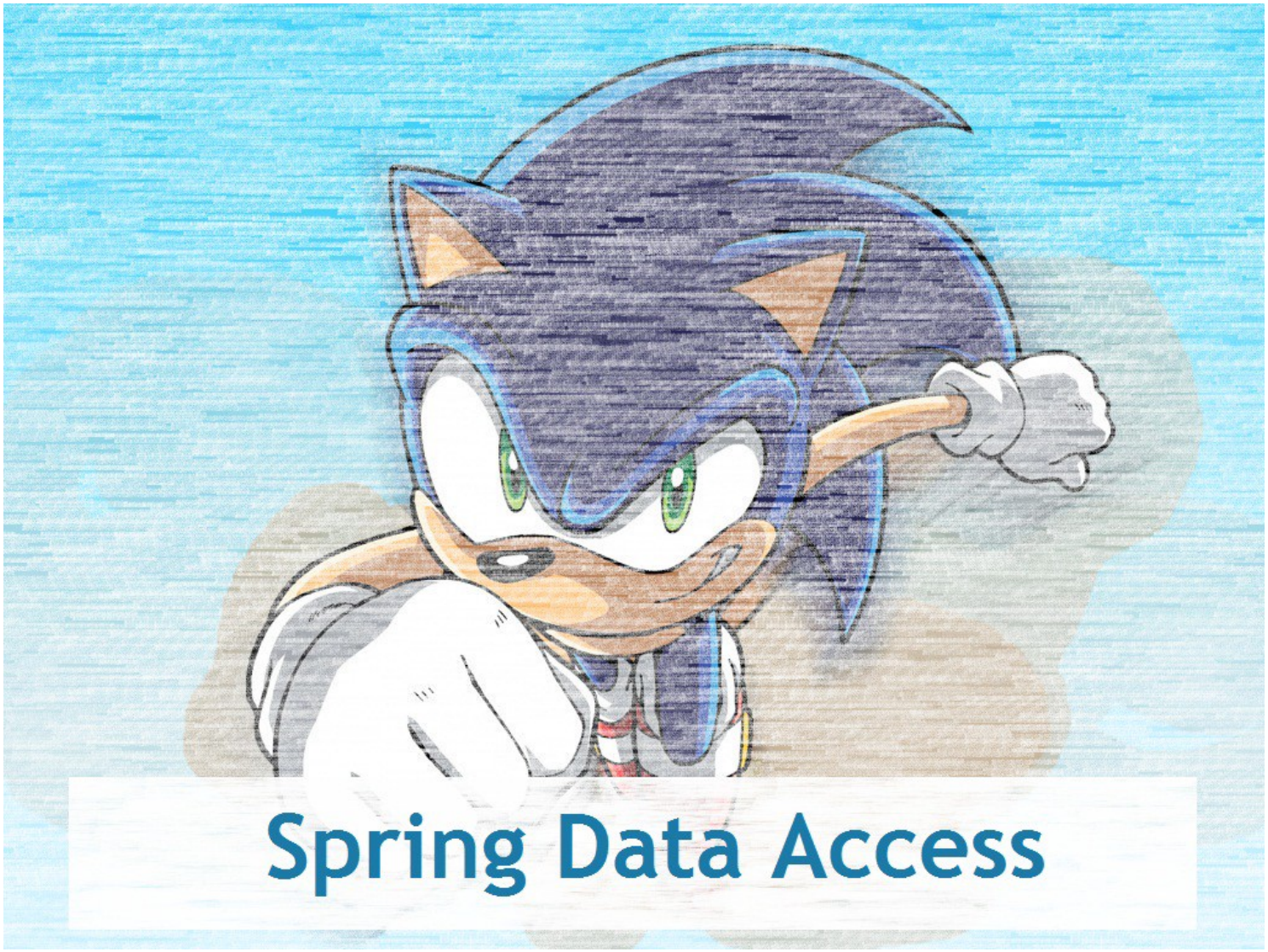
Spring: A Head Start 🔥 — Spring Data Access (Part 6)

Working with databases using JDBC and Hibernate.

Omar Elgabry Follow
Sep 12, 2017 · 8 min read



Spring: A Head Start 🔥 — Spring Data Access

Setting Up The Environment 💪

📢 Before start, make sure you've the following:

1. Database Server (XAMPP, WAMP, MySQL, etc)
2. Hibernate JAR files (and it's dependencies) and JDBC Driver.

If you're using maven, then add the hibernate and JDBC driver (In our example it's MySQL) dependencies in your "pom.xml" file.

```
<!-- Hibernate -->
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.1.7.Final</version>
</dependency>

<!-- MySQL -->
<dependency>
   <groupId>mysql</groupId>
   <artifactId>mysql-connector-java</artifactId>
   <version>5.1.40</version>
</dependency>
```

Once you're done, create *users* table, and generate some fake data for testing.

```
CREATE TABLE `users` (
 `id` int(11) NOT NULL AUTO_INCREMENT,
 `name` varchar(60) NOT NULL,
 `email` varchar(255) NOT NULL,
 PRIMARY KEY (`id`)
)
```

JDBC

Spring JDBC module takes care of all the low-level details starting from opening the connection, prepare and execute the SQL statement, process exceptions, handle transactions and finally close the connection.

All what we need to do is just define the connection parameters and specify the SQL statement to be executed.

But, before using Spring data access modules, let's see how can we make a plain old JDBC.

Basic JDBC Connection

There are basic 5 steps in order to connect and fetch data from a database.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class Test {
  public static void main(String[] args) {
    try {
        // 1. Register JDBC Driver
        Class.forName("com.mysql.jdbc.Driver");

        // 2. Create Connection Object
        Connection con = DriverManager.getConnection("jdbc:mysql://localhost/demoDB","root", "");

        // 3. Create Statement object and Execute a Query
        Statement stmt = con.createStatement();

        // 4. Extract data from result set
        ResultSet rs = stmt.executeQuery("select * from users");
        while(rs.next()) {
          System.out.println(rs.getInt("id") + "  " + rs.getString("name"));
        }

        // 5. Close the connection
        con.close();

        } catch(Exception e){
          // Handle errors here
        }
    }
}
```

So, first, you need to register a JDBC driver so that the driver's class file is loaded and can be used as an implementation of the JDBC interfaces.

Then, we establish a connection by providing the connection URL, username and password.

> *A database URL is an address that points to your database. In our example, `localhost` is the hostname, and `demoDB` is the database name. Different JDBC drivers have different corresponding URLs.*

After that, we create an object of type *Statement* for executing an SQL statement, and returning the result set.

JDBC Connection In Spring MVC

Having a basic idea about how to make a JDBC Connection in Java is essential for building a Web Application in Spring that connects to database.

But, first, let's see how can we structure our application.

— The Structure

- **Data Access Layer (DAO)**: It's commonly used for database interaction. They should expose this functionality through an interface by which the rest of the application will access them. The DAO support in Spring makes it easy to work with data access technologies like JDBC, Hibernate, etc.
- **JDBC Template**: The JDBC Template executes SQL queries, updates statements, stores procedure, etc. It also catches JDBC exceptions.
- **Mappers**: They map between the rows data in a table and java objects. In case of ORMs, like Hibernate, the mapping done using `@Column` on object's field (will be discussed later).
- **Entities**: They're representation for the database tables. Each record in the table represents an entity object. So, entity objects' fields represent table columns.

— JDBC Template & DAO

The JDBC in turn implements the DAO interface to provide the actual implementation code. Though it is not required, you can directly write the implementation code in DAO

class, but not a good practice.

— JDBC Template & DataSource (i.e Connection)

A common practice when using the JDBC Template is to:

1. Configure a *DataSource* bean in your Spring configuration.
2. Dependency-inject that *DataSource* bean in the class implements the DAO.
3. Create a new instance of *JdbcTemplate* in the setter for the *DataSource*.

JDBC — The Process

[1] Create Entities & Their Mappers

```
package com.springmvcdemo;

public class User {
  private Integer id;
  private String name;
  private String email;

  public User(){
  }

  /* getters and setters for objects' fields */
}
```

User.java hosted with ♡ by GitHub view raw

```
package com.springmvcdemo;

import java.sql.ResultSet;
import java.sql.SQLException;
import org.springframework.jdbc.core.RowMapper;

public class UserMapper implements RowMapper<User> {

  @Override
  public User mapRow(ResultSet  rs, int rowNum) throws SQLException {
    User user = new User();
    user.setId(rs.getInt("id"));
    user.setName(rs.getString("name"));
    user.setEmail(rs.getString("email"));
```

```
    return user;
  }
}
```

UserMapper.java hosted with ♡ by GitHub view raw

[2] Create The DAOs Interfaces

List down all the methods for database interaction.

```
package com.springmvcdemo;

import java.util.List;
import javax.sql.DataSource;

public interface UserDAO {

  // set database connection
  public void setDataSource(DataSource ds);

  // create a new user record in the users table
  public void create(String name, String email);

  // get a user with the passed id
  public User getUser(Integer id);

  // get all the users from the users table
  public List<User> listUsers();

  // update a user's email given given the user's id
  public void updateEmail(Integer id, String email);

  // delete a user record from the users table given the user's id
  public void delete(Integer id);
}
```

UserDAO.java hosted with ♡ by GitHub view raw

[3] Create JDBC Implementation Class

Next is the implementation class for the DAO interfaces.

> 💡*The JDBC Implementation Class has an instance of JdbcTemplate which is instantiated by using DataSource object (as a dependency) to do the work.*

> 💡 *You can either inject the DataSource object using XML configurations (Setter Injection), or using Annotation (Field Injection).*

```
package com.springmvcdemo;

import java.util.List;
import javax.sql.DataSource;

import org.springframework.jdbc.core.JdbcTemplate;

public class UserDAOJDBCImpl implements UserDAO {

  private DataSource dataSource;
  private JdbcTemplate jdbcTemplate;

  @Override
  public void setDataSource(DataSource ds) {
    dataSource = ds;
    jdbcTemplate = new JdbcTemplate(dataSource);
  }

  @Override
  public void create(String name, String email) {
    String sql = "insert into users (name, email) values (?, ?)";
    jdbcTemplate.update(sql, name, email);
    System.out.println("Created User Name = " + name + " Email = " + email);
  }

  @Override
  public User getUser(Integer id) {
    String sql = "select * from users where id = ?";
    User user = jdbcTemplate.queryForObject(sql, new Object[]{id}, new UserMapper());
    return user;
  }

  @Override
  public List<User> listUsers() {
    String sql = "select * from users";
    List<User> users = jdbcTemplate.query(sql, new UserMapper());
    return users;
  }

  @Override
  public void updateEmail(Integer id, String email){
```

```
  public void updateEmail(Integer id, String email){
    String sql = "update users set email = ? where id = ?";
    jdbcTemplate.update(sql, id, email);
    System.out.println("Updated Record with ID = " + id );
  }

  @Override
  public void delete(Integer id){
    String sql = "delete from users where id = ?";
    jdbcTemplate.update(sql, id);
    System.out.println("Deleted Record with ID = " + id );
  }
}
```

> *In case of using Annotation for injecting the DataSource. Use `@Autowired` annotation on `jdbcTemplate` field, and remove the `setDataSource()` method (also from UserDAO).*
>
> *The DataSource bean will be injected to JDBC template bean in XML (see next).*

[4] Define Configurations

Define configurations for the *DataSoruce* and JDBC implementing class.

```
<!-- Initialization for data source -->
<bean id="dataSource" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
  <property name="driverClassName" value="com.mysql.jdbc.Driver"/>
  <property name="url" value="jdbc:mysql://localhost/demoDB"/>
  <property name="username" value="root"/>
  <property name="password" value=""/>
</bean>

<!-- Definition for UserDAOJDBCImpl bean (JDBC Implementation Class) -->
<bean id="UserDAOJDBCImpl" class="com.springmvcdemo.UserDAOJDBCImpl">
  <property name="dataSource" ref="dataSource" />
</bean>
```

beans.xml hosted with ♡ by **GitHub** **view raw**

> *In case of using Annotation for injecting the DataSource.*

```
<!-- Initialization for data source -->
<bean id="dataSource" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
  <property name="driverClassName" value="com.mysql.jdbc.Driver"/>
```

```
  <property name="url" value="jdbc:mysql://localhost/demoDB"/>
  <property name="username" value="root"/>
  <property name="password" value=""/>
</bean>

<!-- JDBC Template and Injecting dataSource -->
<bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
  <property name="dataSource" ref="dataSource"></property>
</bean>

<!-- Definition for UserDAOJDBCImpl bean (JDBC Implementation Class) -->
<bean id="UserDAOJDBCImpl" class="com.springmvcdemo.UserDAOJDBCImpl" />
```

beans.xml hosted with ♡ by GitHub view raw

[5] Working With JDBC Implementation Class In Controllers

In order to use the *UserDAOJDBCImpl* class in our controller, we need to inject it in our controller as a dependency, which can be autowired.

> ⚠ *The UserDAOJDBCImpl class has to be defined as a bean so it can be injected.*

Then, we can use it to invoke methods which will interact with the database and return the result.

Finally, you can construct the model object based on the data returned so that it can be displayed in the view pages.

```
package com.springmvcdemo;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;

@Controller
@RequestMapping("/user")
public class UserController {
  @Autowired
```

```
  private UserDAOJDBCImpl userJDBC;

  @RequestMapping(value = "/list", method = RequestMethod.GET)
  public String list(Model model){

  // get all users
  List<User> users = userJDBC.listUsers();

  // add the users to the model
  model.addAttribute("users", users);
  return "list-users";
 }

  @RequestMapping(value = "/newUser", method = RequestMethod.GET)
  public String newUser(Model model){
    model.addAttribute("user", new User());
    return "user-form";
  }

  @RequestMapping(value = "/addUser", method = RequestMethod.POST)
  public String addUser(@ModelAttribute("user") User user){
    String name         = user.getName();
    String email  = user.getEmail();

    // create a new user
    userJDBC.create(name, email);

    return "redirect:/user/list";
  }

  @RequestMapping(value = "/viewUser", method = RequestMethod.GET)
  public String viewUser(@RequestParam("id") int id, Model model){
    // get the user given the user's id
    User user = userJDBC.getUser(id);

    // add user as a model attribute to pre-populate the form
    model.addAttribute("user", user);

    return "user-form";
  }

  @RequestMapping(value = "/deleteUser", method = RequestMethod.GET)
  public String deleteUser(@RequestParam("id") int id, Model model){
    // delete a user
    userJDBC.delete(id);
```

```
    dao.delete(id);

    return "redirect:/user/list";
  }
}
```

[5] The View Pages

One page to display all the users, and the other one to create a new user using an HTML form.

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>

<html>
 <head>
   <title>Spring MVC Demo</title>
 </head>
 <body>
  <h2>List Users</h2>
  <ul>
        <c:forEach var="user" items="${users}">
                <li> ${user.id}, ${user.name}, ${user.email} </li>
        </c:forEach>
   </ul>
 </body>
</html>
```

list-users.jsp hosted with ♡ by GitHub view raw

```
<%@taglib uri="http://www.springframework.org/tags/form" prefix="form"%>

<html>
 <head>
   <title>Spring MVC Demo</title>
 </head>
 <body>
  <h2>User Data</h2>
   <form:form action="addUser" modelAttribute="user" method="POST">

     <form:label path="name">Name</form:label>
     <form:input path="name" />

     <form:label path="email">Email</form:label>
     <form:input path="email" />

     <input type="submit" value="Submit"/>
```

```
    </form:form>
  </body>
 </html>
```

Hibernate

Hibernate is an ORM framework that maps between java objects and database records. It acts as a middle layer between our application code and the database.

It handles the low level SQL code, minimize the amount of JDBC code, and provides object-To-Relational-Mapping (ORM). Actually, Hibernate uses JDBC in the background for database communications. So, It's just a layer of abstraction on top of JDBC.

All what we to do is to tell Hibernate how our java object maps to a database table. This is done inside the object class (or in XML configuration). Hibernate will then get, store, update, or delete this object in the table based on the mappings we've defined.

A quick examples on how Hibernate makes it a lot easier … 🏃

```
// Saving an object
User newUser = new User("John", "Doe", "myemail.com");

// "session" is a special Hibernate object
int thePrimaryId = (Integer) session.save(newUser);

// Retrieve an object
User user = session.get(User.class, thePrimaryId);

// Quering object
// We pass what's called "Hibernate Query Language" (will be
discussed later)
Query query = session.createQuery("from user");
List<User> usersList = query.list();
```

Hibernate — The Process

[1] The Hibernate Configuration file

It tells hibernate how to connect to a database, which is the JDBC configuration, since it uses JDBC to connect to database. So, create a "hibernate.cfg.xml" file in "src" folder.

```xml
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
  <session-factory>
    <!-- JDBC Database connection settings -->
    <property name="connection.driver_class">com.mysql.jdbc.Driver</property>
    <property name="connection.url">jdbc:mysql://localhost/demoDB</property>
    <property name="connection.username">root</property>
    <property name="connection.password"></property>

    <!-- JDBC connection pool settings ... using built-in test pool -->
    <property name="connection.pool_size">1</property>

    <!-- Select our SQL dialect -->
    <property name="dialect">org.hibernate.dialect.MySQLDialect</property>

    <!-- Echo the SQL to stdout -->
    <property name="show_sql">true</property>

    <!-- Set the current session context -->
    <property name="current_session_context_class">thread</property>

  </session-factory>
</hibernate-configuration>
```

hibernate.cfg.xml hosted with ♡ by GitHub view raw

This file has the following:

[1] Session Factory

It reads the hibernate configuration file, gets a connection to database, and creates the session objects, and it's created once during the application.

> 💡*A session is a wrapper around the JDBC connection. It's the main object to save and retrieve objects to the database.*
>
> *It's short-lived object, so, for a given method, you use a session, and then trow it away, and so on. It's created, and retrieved from the Session Factory.*

[2] JDBC Database connection settings: These are the settings for JDBC connections such as JDBC driver, connection URL, username and password.

[3] Connection Pool: For simplicity, keep it 1 for now. It's the max number of connections in the connection pool.

> *Hibernate make use of connection pool for connecting to the database.*
>
> *Hibernate provides an internal connection manager, which is rudimentary. For the best performance and stability, use a third-party tool ('ll do that later).*

[4] SQL Dialect: It's the SQL dialect your database uses. So, it tells Hibernate to generate the appropriate SQL statements for the chosen database.

[5] Show SQL: To print out the SQL used to communicate to the database.

[6] Session Context: The default context is `thread` which means the session factory will bind the session to the thread from which `openSession()` is called.

This is useful because you can later call `sessionFactory.getCurrentSession()` which will return the session that is bound to the currently running thread.

[2] Annoate Java Classes (Entities)

An entity is a plain java class that mapped to database table. We use annotation to do the mappings. This is what's called Object Relational Mapping (ORM).

> *There's another way to define the mappings usign XML configuration file, which is the old way.*

So, **first**, map a class to database table. **Then**, map the fields to database table columns.

```
package com.springmvcdemo;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="users")  // The default table name is the class name
public class User {

  @Id                    // Identifies a primary key.
```

```
  @Column(name="id")    // The default name is the name of the field
  private Integer id;

  @Column(name="name")
  private String name;

  @Column(name="email")
  private String email;

  public User(){
  }

  public User(Integer id, String name, String email) {
    this.id = id;
    this.name = name;
    this.email = email;
  }

  /* getters and setters for objects' fields */
}
```

> ⚠ *Don't forget to define constructors, getters and setters.*

[3] Setup Session Factory And Create A Session Object

The session factory to create a session object which in turn will be used to interact with the database.

First, we load the configuration file, the default name configuration file is "*hibernate.cfg.xml*". Then, the class(es) to be scanned for mapping, and we create the session object.

Finally, close the factory session. It destroys the *SessionFactory* and release all resources (caches, connection pools, etc).

```
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class Test {
  public static void main(String[] args) {
    SessionFactory factory = new Configuration()
```

```
            .configure("hibernate.cfg.xml")
            .addAnnotatedClass(User.class)
            .buildSessionFactory();

    Session session = factory.getCurrentSession();
    try {
      // your database code
    } catch(Exception e){

    } finally {
      factory.close();
    }
  }
}
```

Test.java hosted with ♡ by GitHub view raw

[4] Start A Transaction

In Hibernate, you always use a database transaction when interacting with the database. Regardless if it's a create, read, update, or delete operation. Database transactions are never optional in Hibernate.

```
// start a transaction
session.beginTransaction();

// database operations
// ... We can wrap a set of queries inside a transaction

// commit transaction
session.getTransaction().commit();
```

The `commit()` statement indicates the end of the transaction, and commits the changes to the database. No changes will be applied to the database unless the transaction is committed.

Once you committed the transaction, you can't use the session object anymore. You need to re-assign the session object to get a new session, and begin a transaction again.

```
session = factory.getCurrentSession();
```

[4] Save (Create) Object To Database

For every operation, don't forget to get a new session object using `factory.getCurrentSession()`, start the transaction, and commit.

```
// create a user object
User newUser = new User(125, "Alex", "alex@example.com");

// save the user object
session.save(newUser);
```

[5] Reading Objects

```
// retrieve student based on the id: primary key
User user = session.get(User.class, newUser.getId());
```

There is a query language in Hibernate called **HQL** used to query the database. In HQL, we use the class name and property names, instead of table and column names.

```
// "User" is the name of the class
List<User> users = session.createQuery("from User").list();

// "u" is an alias that maps to the actual java object (user),
// where u.name is a property of user object
List<User> users = session.createQuery("from User AS u where
u.name='Alex'").list();
```

> ⚠ *In Hibernate 5.2 or higher, the Query `list()` method has been deprecated, use `getResultList()` instead.*

[6] Updating Objects

We can update user object using **setters**.

> 💡 *Since a persistent instance of the user object is returned by `get()` method (after being fetched from the table), updating this user object's field value will also reflect on the table values.*

```
// retrieve user based on the id: primary key
int userId = 125;
User user = session.get(User.class, userId);

// update name to "Max"
user.setName("Max");
```

Or, we can write the query to update specific user(s).

```
session.createQuery("update User set name='Max' WHERE
id=125").executeUpdate();
```

[7] Deleting Objects

The same idea in update applies here; We can either delete a persistent instance of the user object returned by `get()` method, or, write and execute a query to do so.

```
// retrieve user based on the id: primary key
int userId = 125;
User user = session.get(User.class, userId);

// delete the student
session.delete(user);

// delete user id=125 (using query)
session.createQuery("delete from User WHERE id=125").executeUpdate();
```

Thank you for reading! If you enjoyed it, please clap 👏 for it.

Java Spring Framework Programming Software Development Other

About Help Legal

Get the Medium app

Download on the App Store

GET IT ON Google Play